data-pipelining-with-polygon

November 12, 2026

0.1 Post Updates

Update 11/6/2026: Merged `polygon_fetch_full_history` and `polygon_pull_data` into a single module, `polygon_pull_equity_data`, and renamed `polygon_pull_data` to `polygon_pull_equity_data`. The pull now also fetches Massive's dividend adjustment factors and applies them to the split-adjusted prices, adding `adj_open`, `adj_high`, `adj_low`, `adj_close`, and `adj_vwap` columns. That was the one item under Future Investigation, so that section is gone, and a new section below covers the dividend adjustment. Also folded `polygon_month_end` and `polygon_quarter_end` into the same module as a single function, `polygon_resample_period_end`, which takes a `period` argument of either `month` or `quarter`. Switched the examples from AMZN to AAPL, since AMZN doesn't pay a dividend and AAPL shows the adjustment at work.

Update 8/6/2026: Brought the post back in line with the current source. Polygon.io has rebranded to Massive and the Python client is now imported as `massive`, `polygon_pull_data` gained a `parquet_export` parameter and returns the resulting DataFrame, and `polygon_fetch_full_history` renamed its `full_history_df` parameter to `existing_history_df`. Replaced the reproduced docstrings with prose descriptions, added the free tier constraints that shape the design, and documented the on-disk layout the data lands in.

0.2 Introduction

Where the previous post on collecting crypto asset data from Coinbase covered digital assets, this one covers the equity side. The two functions below pull minute, hour, and daily bars for equities and ETFs from Polygon.io, since rebranded as Massive.

Two constraints from the free tier drive most of the design. The first is a two year limit on historical data, which is why the examples below start from a date two years back rather than attempting to pull an asset's full history. The second is a rate limit of five API requests per minute, which is what the `free_tier` flag handles. When using the free tier, the fetch pauses for twelve seconds between requests. A single API call also returns at most 5,000 records, so any meaningful date range has to be walked through in chunks and reassembled.

The work lives in a single module, `polygon_pull_equity_data`, and is split across functions that build on one another. `polygon_fetch_full_history` walks a date range and returns it as a single dataframe. `polygon_pull_equity_data` wraps that to maintain pickle, Excel, and Parquet files on disk, extending an existing record with the rows it is missing, or creating a new record where none exists. It also adds dividend-adjusted prices, using two helpers in the same module: `polygon_fetch_dividend_factors` and `apply_dividend_adjustment`.

0.3 Python Imports

[1]:
```python
# Standard Library
import os
import sys
import time
import warnings

from datetime import datetime
from pathlib import Path

# Data Handling
import pandas as pd

# Suppress warnings
warnings.filterwarnings("ignore")

# Add src/ to the path so settings.py and the helper modules can be imported
src_directory = str(Path.cwd().parents[2] / "src")
if src_directory not in sys.path:
    sys.path.append(src_directory)

from settings import config

DATA_DIR = config("DATA_DIR")  # keep only the directories this post uses
```

0.4 Python Functions

Here are the modules needed for this project:

- polygon_pull_equity_data: Read existing data file, download price data from Polygon, apply dividend adjustment factors, and export data. The module also holds the helpers `polygon_fetch_full_history`, `polygon_fetch_dividend_factors`, and `apply_dividend_adjustment`.
- polygon_resample_period_end: Resample the daily data on disk to month-end or quarter-end closes.

[2]:
```python
from polygon_pull_equity_data import (
    polygon_fetch_full_history,
    polygon_pull_equity_data,
)
from polygon_resample_period_end import polygon_resample_period_end
```

One naming note before the code: Polygon.io has rebranded to Massive, and the official Python client followed. The package is now `massive` rather than `polygon-api-client`, so the import below reads `from massive import RESTClient`. The API surface is otherwise unchanged, and I've kept "Polygon" in the function names and directory layout since that's what the existing data is filed under.

0.5 Function Usage

Both examples below use daily bars, but switching to minute or hour data is a matter of changing the `timespan` and `multiplier` arguments, with everything else unchanged. Note that finer granularity burns through the free tier rate limit considerably faster, since each request covers a much shorter window. Massive aggregates minute data into hour bars, so the time to pull minute and hour data is actually the same. Both functions accept only `minute`, `hour`, or `day` for `timespan` and raise an exception otherwise, since the window sizing logic is only defined for those three.

0.5.1 Polygon Fetch Full History

This is the lower level of the two functions. Starting from `current_start`, it walks forward through the date range in fixed size windows - 180 days for daily data, 15 days for minute and hour data - requests each window from the API, and appends the results to the dataframe passed in as `existing_history_df`, stopping once it reaches the present. Because each request is capped at 5,000 records, ranges spanning years are handled the same way as the single window below: the individual responses are concatenated and returned as one dataframe.

The API key is read through `config`, which merges everything in the `.env` file into the settings defaults, so `config("POLYGON_KEY")` resolves without any further setup:

[3]:
```
from massive import RESTClient

# Load API key from the environment variables
POLYGON_KEY = config("POLYGON_KEY")

# Get the environment variable for where data is stored
DATA_DIR = config("DATA_DIR")

# Open client connection
client = RESTClient(api_key=POLYGON_KEY)

# Create an empty DataFrame
df = pd.DataFrame({
    'Date': pd.Series(dtype="datetime64[ns]"),
    'open': pd.Series(dtype="float64"),
    'high': pd.Series(dtype="float64"),
    'low': pd.Series(dtype="float64"),
    'close': pd.Series(dtype="float64"),
    'volume': pd.Series(dtype="float64"),
    'vwap': pd.Series(dtype="float64"),
    'transactions': pd.Series(dtype="int64"),
    'otc': pd.Series(dtype="object")
})

# Example usage - daily
df = polygon_fetch_full_history(
    client=client,
    ticker="AAPL",
```

```
    timespan="day",
    multiplier=1,
    adjusted=True,
    existing_history_df=df,
    current_start=datetime(2025, 1, 1),
    free_tier=True,
    verbose=False,
)

time.sleep(12)  # Sleep for 12 seconds to avoid hitting rate limits
```

The example above pulls daily data starting from 1/1/2025. Here are the first five rows:

[4]:
```
display(df.head())
```

```
                 Date    open      high       low   close      volume  \
0 2025-01-02 05:00:00  248.93  249.1000  241.8201  243.85  55740731.0
1 2025-01-03 05:00:00  243.36  244.1800  241.8900  243.36  40244114.0
2 2025-01-06 05:00:00  244.31  247.3300  243.2000  245.00  45045571.0
3 2025-01-07 05:00:00  242.98  245.5500  241.3500  242.21  40855960.0
4 2025-01-08 05:00:00  241.92  243.7123  240.0500  242.70  37628940.0

       vwap  transactions   otc
0  244.3303        685448  None
1  243.0805        493786  None
2  245.1726        493920  None
3  242.9459        462887  None
4  242.2995        406916  None
```

0.5.2 Polygon Pull Equity Data

This function wraps the one above and performs the following:

1. Attempt to read an existing pickle data file.
2. If a data file exists, then pull updated data starting from the last date on record.
3. Otherwise, pull all historical data available for that asset for the past 2 years (or chosen interval).
4. If `adjusted` is set, pull the dividend adjustment factors and add the `adj_open`, `adj_high`, `adj_low`, `adj_close`, and `adj_vwap` columns (covered in the next section).
5. Store Pickle, Excel, and/or Parquet files of the data in the specified directories.
6. Return the resulting dataframe.

Setting `force_existing_check` re-pulls the entire range rather than starting from the last date on record, which is how gaps in an existing file get filled.

The `base_directory`, `source`, `asset_class`, and `timespan` arguments together tell the function where in the local filesystem to look for an existing pickle file, and where to store the updated exports. They assemble into a path of the form `{base_directory}/{source}/{asset_class}/{timespan}/{ticker}`, so the example below reads and writes:

```
Data/Polygon/Equities/day/AAPL.pkl
Data/Polygon/Equities/day/AAPL.xlsx
Data/Polygon/Equities/day/AAPL.parquet
```

Keeping the timespan in the path means minute, hour, and daily records for the same ticker sit side by side without overwriting one another.

[5]:
```python
current_year = datetime.now().year
current_month = datetime.now().month
current_day = datetime.now().day

# Example usage - daily
df = polygon_pull_equity_data(
    base_directory=DATA_DIR,
    ticker="AAPL",
    source="Polygon",
    asset_class="Equities",
    start_date=datetime(current_year - 2, current_month, current_day),
    timespan="day",
    multiplier=1,
    adjusted=True,
    force_existing_check=True,
    free_tier=True,
    verbose=False,
    excel_export=True,
    pickle_export=True,
    parquet_export=True,
    output_confirmation=True,
)

time.sleep(12)  # Sleep for 12 seconds to avoid hitting rate limits
```

```
File found…updating the AAPL day data.
Last date in existing data: 2026-11-11 05:00:00
Number of rows in existing data: 1257
Forcing check of existing data…

Pulling dividend adjustment factors for AAPL…
Number of dividends found for AAPL: 58
Exporting AAPL day data to Excel…

Exporting AAPL day data to Pickle…
Exporting AAPL day data to Parquet…
The first and last date of day data for AAPL is:

                 Date   open     high       low   close      volume     vwap  \
0 2021-11-09 05:00:00  150.2  151.428  150.0601  150.81  56787930.0  150.792

   transactions   otc    adj_open    adj_high     adj_low   adj_close  \
0        455006  None  146.603161  147.801754  146.466611  147.198553
```

```
     adj_vwap
0  147.180984

                     Date     open    high      low   close        volume  \
1256 2026-11-11 05:00:00  349.545  355.91  349.325  355.63  3.445348e+07

          vwap  transactions   otc    adj_open    adj_high     adj_low  \
1256  354.3937        684164  None  349.279695  355.639864  349.059862

       adj_close    adj_vwap
1256  355.360077  354.124715

Number of rows after data update: 1257
Number of rows added during update: 0
Polygon data complete for AAPL day data.
--------------------
```

And here are the first five rows of the returned dataframe:

[6]:
```
display(df.head())
```

```
                 Date    open     high       low   close      volume  \
0 2021-11-09 05:00:00  150.20  151.428  150.0601  150.81  56787930.0
1 2021-11-10 05:00:00  150.02  150.130  147.8500  147.92  65160017.0
2 2021-11-11 05:00:00  148.96  149.430  147.6810  147.87  40699950.0
3 2021-11-12 05:00:00  148.43  150.400  147.4800  149.99  63804006.0
4 2021-11-15 05:00:00  150.37  151.880  149.4300  150.00  59182803.0

       vwap  transactions   otc    adj_open    adj_high     adj_low  \
0  150.7920        455006  None  146.603161  147.801754  146.466611
1  148.6034        544210  None  146.427471  146.534837  144.309436
2  148.1690        338584  None  145.392855  145.851600  144.144483
3  149.3536        404421  None  144.875547  146.798371  143.948296
4  150.2881        422055  None  146.769090  148.242930  145.851600

    adj_close    adj_vwap
0  147.198553  147.180984
1  144.377760  145.044794
2  144.328957  144.620797
3  146.398189  145.777029
4  146.407950  146.689151
```

Note that `Date` is returned as a regular column with a plain integer index rather than as the index itself. That is deliberate, and any downstream analysis can call `set_index("Date")` as needed.

0.5.3 Dividend Adjustment

Setting `adjusted=True` makes the API adjust the bars for splits, but not for dividends. To get total return prices, `polygon_pull_equity_data` makes one more request, to Massive's

dividends endpoint (`client.list_stocks_dividends`). Each dividend record there carries a `historical_adjustment_factor`, and Massive documents how to use it: for a price on date D, find the first dividend whose ex-dividend date falls after D, and multiply the price by that dividend's factor. The factors are already cumulative, so no running product is needed, and bars on or after the most recent ex-dividend date get a factor of 1.

`apply_dividend_adjustment` does this lookup with a forward `merge_asof` and multiplies the split-adjusted open, high, low, close, and VWAP by the result to create `adj_open`, `adj_high`, `adj_low`, `adj_close`, and `adj_vwap`. A few details are worth noting:

- Every new dividend changes the factor for all of the history before it, so the adjusted columns are recomputed over the full record on every run rather than only for the newly appended rows.
- Volume gets no dividend adjustment. A cash dividend doesn't change the number of shares outstanding or traded, so only the split adjustment the API already applies is needed.
- The timestamps are in UTC, so each bar is first mapped to its trading date in US/Eastern time. Otherwise an after-hours minute or hour bar, which falls after midnight UTC, would be counted as part of the next session.

If `adjusted=False`, the prices aren't split-adjusted, so applying the factors would be wrong across any split. In that case the function skips the adjusted columns and prints a note saying so.

Here is the ratio of `adj_close` to `close` on the last bar before each recent ex-dividend date, which is where the factor steps up:

[7]:
```
ratio = (df["adj_close"] / df["close"]).round(6)
display(df.loc[ratio.ne(ratio.shift(-1)), ["Date", "close", "adj_close"]].
  ↪assign(factor=ratio).tail(8))
```

	Date	close	adj_close	factor
814	2025-02-07 05:00:00	227.63	225.818293	0.992041
877	2025-05-09 04:00:00	198.53	197.166496	0.993132
939	2025-08-08 04:00:00	229.35	228.073667	0.994435
1003	2025-11-07 05:00:00	268.47	267.278799	0.995563
1064	2026-02-06 05:00:00	278.12	277.154367	0.996528
1127	2026-05-08 04:00:00	293.32	292.575261	0.997461
1189	2026-08-07 04:00:00	313.33	312.822405	0.998380
1256	2026-11-11 05:00:00	355.63	355.360077	0.999241

0.5.4 Polygon Resample Period End

Monthly and quarterly analysis doesn't need daily bars, so this function makes a lower-frequency copy of a record already on disk. It makes no API calls. It reads the existing pickle file, keeps `close` (plus `adj_close` when the file has it), and takes the last value in each month or quarter. The `period` argument chooses between the two: `month` resamples to month-end and `quarter` resamples to quarter-end. Any other value raises an exception.

The other arguments work the same as above, with `timespan` naming the record to read. The results go in their own directories alongside the daily data, so the two examples below write:

`Data/Polygon/Equities/Month_End/AAPL_ME.pkl`

```
Data/Polygon/Equities/Month_End/AAPL_ME.xlsx
Data/Polygon/Equities/Month_End/AAPL_ME.parquet
Data/Polygon/Equities/Quarter_End/AAPL_QE.pkl
Data/Polygon/Equities/Quarter_End/AAPL_QE.xlsx
Data/Polygon/Equities/Quarter_End/AAPL_QE.parquet
```

[8]:
```python
# Example usage - month end
df_month_end = polygon_resample_period_end(
    base_directory=DATA_DIR,
    ticker="AAPL",
    source="Polygon",
    asset_class="Equities",
    timespan="day",
    period="month",
    excel_export=True,
    pickle_export=True,
    parquet_export=True,
    output_confirmation=True,
)

# Example usage - quarter end
df_quarter_end = polygon_resample_period_end(
    base_directory=DATA_DIR,
    ticker="AAPL",
    source="Polygon",
    asset_class="Equities",
    timespan="day",
    period="quarter",
    excel_export=True,
    pickle_export=True,
    parquet_export=True,
    output_confirmation=True,
)
```

```
Month end data complete for AAPL
--------------------
Quarter end data complete for AAPL
--------------------
```

Unlike the daily data, the result is indexed by the period-end date. Here are the last five month-end rows:

[9]:
```python
display(df_month_end.tail())
```

Date	close	adj_close
2026-07-31	308.91	308.409566
2026-08-31	316.85	316.609511
2026-09-30	333.02	332.767238

```
2026-10-31  344.63  344.368426
2026-11-30  355.63  355.360077
```

And the last five quarter-end rows:

[10]:
```
display(df_quarter_end.tail())
```

```
             close   adj_close
Date
2025-12-31  271.86  270.916102
2026-03-31  253.79  253.145627
2026-06-30  289.36  288.891237
2026-09-30  333.02  332.767238
2026-12-31  355.63  355.360077
```

0.6 References

1. Massive (formerly Polygon.io)
2. Massive REST API quickstart
3. massive on PyPI
4. massive-com/client-python on GitHub
5. Reusable & Extensible Python Functions For Financial Data Analysis